

TRIPLY

Complete Technical Analysis & Backend Plan

Verified against source on 2026-10-09 | Frontend-only analysis, no source changes

1. Architecture Verdict

Triply is MVVMW + Cubit almost everywhere, with three deliberate exceptions: (a) both auth features use Provider + ChangeNotifier (AuthProvider, TourGuideAuthProvider) as the global/local session layer; (b) tour_guide/onboarding and role_selection use local setState / stateless only because no shared state exists; (c) tourist/checkout keeps a thin CheckoutController compatibility wrapper delegating to CheckoutCubit, because the frozen booking_public feature also instantiates it.

Views are thin, Cubits own state and business logic, models are fromJson-capable data classes, and data comes from static mock lists plus three session stores (GuideTripsStore, MyBookingsPublic, GuideTripDetailsSource._overrides) plus Firebase Auth session and the Firestore users collection. The architecture is consistent across the project except for auth (Provider) and the two stateless screens.

2. Tourist Flow

App start: main.dart runs Firebase.initializeApp + dotenv.load, then an auth gate (Consumer<AuthProvider>) shows a spinner while initial, otherwise the RoleSelectionScreen. Tourist card -> OnboardingScreen (3 slides -> SignInScreen).

Authentication

- Login (sign_in_screen.dart): email or Google via AuthProvider.signIn/signInWithGoogle -> pushReplacementNamed(/home). Error snackbars from AuthRepository.getErrorMessage.
- Create Account: name, Egyptian-validated phone (^1[0125]d{8} normalized to +20), email, password -> AuthRepository.signUpWithEmail creates the Firebase user, sets displayName, and best-effort writes users/{uid} {fullName, email, phone, createdAt} -> /home.
- Forgot password: sendPasswordResetEmail -> snackbar + pop (Firebase email link, no in-app code screen). new_password_screen handles oobCode via confirmPasswordReset.

Discovery

- HomeScreen (HomeCubit): places/guides/trips sections from mock_places, mock_guides, mock_trips_raw. Bottom nav (AppBar, push-based): Home / My Trips / Places(Map) / Community / Profile.
- SearchScreen (SearchCubit): case-insensitive partial match over place name/city/category/address in mockPlaces; recent searches, filter chips (SearchFilter), popular lists by id.
- MapScreen (MapCubit): flutter_map + OpenStreetMap tiles, markers from mockPlaces lat/lng, search + category filter chips, preview card -> PlaceDetailScreen(placeId).
- PlaceDetailScreen (PlaceDetailCubit): place + guides + trips + stories tabs, favorite toggle (setState + cubit emit).
- GuidesListScreen (GuidesCubit: search + language filter) -> GuideProfileScreen (trips, Book button). Trips/popular lists over mockTripsJson.

Booking (public trip)

TripDetailsScreen -> Book Now -> AppRoutes.booking(trip) -> BookingPublicScreen (BookingPublicCubit): summary card + SeatsStepper clamped to capacity - peopleCount + requests + 5% fee breakdown -> Confirm & Pay runs CheckoutController.startPayment (Paymob 3-step: auth -> order -> payment key, webview) -> on success cubit.confirm records MyBookingsPublic.record(tripId, seats) -> BookingConfirmedScreen -> MyTripsScreen.

Booking (private guide)

Guide profile -> BookingScreen(guidId) (BookingCubit, 4 steps: place -> date/time/duration -> travelers/meeting/notes -> confirm with guide.currency x hours x travelers + 5% fee) -> same Paymob checkout -> MyBookingsPublic.recordPrivate (key pvt-{guidId}_{ms}) -> My Trips.

My Trips / Profile / Community / SOS

- MyTripsScreen (MyTripsCubit): Upcoming/Ongoing/Completed/Cancelled tabs over booked records only; rows show recorded seats/totalPaidFor (never mock headcounts); delete button removes the record + themed refund snackbar (5s).
- ProfileScreen (ProfileCubit + Firebase user): stats (trips count from live bookings), language selector (state-only), Privacy screen (real Firebase reset flow), Emergency screen (static 123/122/180 with copy + tel: dialer), real logout -> RoleSelectionScreen.
- CommunityScreen (CommunityCubit): post/story create + own-delete, like toggles, story viewer, guide-profile navigation.
- NotificationsScreen: mock list with mark-all/one-read.

3. Tour Guide Flow

- Onboarding (3 slides, setState only) -> TourGuideLoginScreen. Login: signInGuide returns role; verified guides (users/{uid}.verification.reviewStatus == approved) go straight to GuideDashboardScreen, others to verification; tourist-role accounts redirect to tourist Home.
- Signup (+20 phone, licenseNumber, [Arabic, English]) -> Verification screen: ID + license picked via FilePicker (jpg/png/pdf, 10MB) -> CloudinaryService.uploadFile x2 (preset triply_tour_guide_info, folder triply/guide_verifications/{uid}) -> submitVerificationDocuments (reviewStatus pending, canPublishTrips false) -> dashboard. Skip-for-now also lands on dashboard; back returns to login.
- GuideDashboardScreen (DashboardCubit): stats recomputed from GuideTripsStore, verification banner (hidden once approved), booking requests -> BookingDetailsScreen, bell -> notifications, avatar -> own profile (guidId g2). Bottom nav Home/My Trips/Profile.
- GuideTripsScreen (GuideTripsCubit, status tabs active/pending/rejected/completed/cancelled) -> Create (CreateTripCubit form incl. gallery cover as local path) -> submitTrip saves GuideTrip + full GuideTripDetails override -> Pending tab -> View/Edit (prefilled, saves persist for session) -> TravelersScreen -> EarningsScreen (mock EGP).
- Profile (ProfileTgCubit): Firebase name/photo first, Firestore phone/about/location with mock fallbacks; Availability/Notifications/Settings sub-screens; logout -> RoleSelectionScreen.

Missing backend needs

- Verification approval is manual/absent: status stays pending forever in mock; no approver exists.
- Earnings and travelers are static mocks; created trips, bookings and edits live in static session stores cleared on restart.

4. Admin Flow

Entry ONLY via the role-selection footer ("Triply Admin Portal") -> AdminLoginView / AdminCreateAccountView (AdminAuthCubit; demo rule: identifier must contain "triplykl", case-insensitive, else "You are not allowed to access the Admin Dashboard."). Success pushes HomeView wrapped in BlocProvider(AdminAuthCubit.authenticated(session)). Sessions are in-memory (lost on restart). The d_auth_provider.dart stub and d_app_routes.dart duplicate AppRoutes class are unused leftovers.

- Shell (HomeView): desktop permanent sidebar / mobile drawer, 8 indexed pages (Overview, Trips, Users, Bookings, User Posts, Guides, Trip Applications, App Settings). Header subtitle + sidebar footer show the session name. Logout asks "Are you sure you want to logout?" (Cancel stays, Logout clears session + pushAndRemoveUntil RoleSelectionScreen).
- Overview: hardcoded stat cards + Pending Actions + guide approve/reject/suspend on a local mock-guides copy.
- Users/Bookings/Trips/Posts/Applications: search-filtered mock tables (case-insensitive, partial, empty states), view dialogs, suspend/cancel/remove actions on local copies. Bookings dialog shows tourist/trip/guide/date/price/total/status/type.
- Guides: approve/reject/suspend + details dialog (email, phone, city, submitted date, document box, uploaded trips with a real info dialog). Trip Applications: approve/reject-with-reason + review dialog (image, itinerary, participants).
- Settings: local commission field + approval-mode dropdown + saved snackbar.
- Overflow hardening: ellipsis/maxLines on table cells, maxWidth-constrained scrollable dialogs, responsive overview split and card margins.

5. Cross-Role Connections

ACTUAL (verified, not assumed): the three roles are runtime silos. They share only the Firebase Auth session, the users/{uid} document shape, and static mock files. Guide-created trips live in the in-memory GuideTripsStore - invisible to tourists (who read mockTripsJson) and to Admin (who reads d_mock_*). Tourist bookings live in MyBookingsPublic statics - invisible to guides and Admin. Verification documents reach Cloudinary + the Firestore verification map, which Admin never reads (admin approve buttons mutate local mock copies).

Required backend relationships

Relationship	Cardinality	Enforced today?
User - TouristProfile	1 - 1	No (same Firebase user)
User - GuideProfile	1 - 1	No (same Firebase user)
Guide - Trip	1 - N	No (mock ids only)
Trip - Place	N - 1	No (strings, not FKs)
Tourist - Booking - Trip	N - N via Booking	No (session map)
Booking - Guide	N - 1	No
User - Post/Story	1 - N	No (local lists)
Trip - Review	1 - N	Partial (fields only)

6. Current Logic / Data Flow

User Action

- > View (thin, BlocBuilder / setState for local UI)
- > Cubit (state + business logic + validation)
- > Model.fromJson (typed data classes)
- > static mock list / session store / Firebase / Cloudinary
- > emit(State) -> BlocBuilder UI

Exceptions: auth Providers (ChangeNotifier + Repository over Firebase), checkout wrapper, admin pages (local setState over mock copies).

Business-logic homes

- Pricing: PriceCalculator (5% fee) reused by booking cubits; private rate = guide.pricePerHour x hours x travelers.
- Validation: Egyptian phone regex, Luhn-gated card checks (sandboxMode=true), 10MB file cap, 6-char passwords, capacity-peopleCount seat clamp.
- Booking lifecycle today: Available -> Pending (recorded at confirm) -> Completed/Cancelled (mock statuses); no payment-state machine beyond Paymob token success/failure.
- Search: local case-insensitive substring match; admin tables filter the same way.

7. Backend Target Architecture

Recommended: NestJS + PostgreSQL. Typed DTOs mirror the Dart models 1:1; relational integrity fits the booking/guide/trip graph; mature RBAC middleware for the three roles. Keep Firebase Auth as the identity provider (verify ID tokens server-side, map to users rows) ~~do not~~ rebuild login.

```
Flutter: View -> Cubit -> Repository -> RemoteDataSource -> ApiClient (Dio)
Backend: Controller -> Service -> Repository -> PostgreSQL
Files/Media: signed-URL upload via backend (Cloudinary stays as storage)
```

Flutter work per feature (strangler pattern): add Repository interface + RemoteDataSource behind each Cubit, keep the mock list as fallback source, flip one feature at a time. Model.fromJson shapes already match ~~XXXX~~ API responses, so no model rewrites.

8. Database Plan

PostgreSQL recommended: relational booking integrity (capacity checks in transactions), JSONB for flexible itinerary/highlights, trigram search on place/trip names, mature NestJS/TypeORM tooling.

Table	Key fields	Relationships
users	id PK, email UNIQUE, phone, role[tourist guide admin], display_name, avatar_url	1-1 tourist_profiles, guide_profiles
tourist_profiles	user_id FK UNIQUE	1-1 users
guide_profiles	user_id FK UNIQUE, specialty, rating	1-1 users; 1-N trips, verifications
guide_verifications	guide_id FK, id_doc, license_doc, status, reviewer, decided_at	N-1 guides
places	id PK, name, city, category, lat/lng, rating	1-N trips
trips	id PK, guide_id FK, place_id FK, price, capacity, status	N-1 guides/places; 1-N bookings
bookings	id PK, tourist_id, trip_id, guide_id FKs, seats, total, status, UNIQUE(tourist,trip,date)	N-1 all three
posts/post_images/stories	user_id FK cascade on delete	N-1 users
notifications	user_id FK, type, read, created_at	N-1 users
reviews	trip_id + tourist_id UNIQUE, rating	N-1 trips/users
payments	booking_id FK UNIQUE, paymob refs, amount	1-1 bookings

9. API Plan (grouped by role)

Auth

POST /auth/register | POST /auth/login | POST /auth/logout | POST /auth/refresh | POST /auth/password-reset

Tourist

GET /places?q=&category= | GET /trips | GET /trips/:id | POST /bookings | GET /bookings/my | PATCH /bookings/:id/cancel
POST /posts | DELETE /posts/:id | POST /stories | DELETE /stories/:id | GET /guides | GET /guides/:id

Guide

GET /guide/trips | POST /guide/trips | PUT /guide/trips/:id | GET /guide/bookings | GET /guide/earnings | GET /guide/travelers?trip= | POST /guide/verification | PATCH /guide/profile

Admin (all behind role=admin middleware)

GET /admin/users | GET /admin/guides?status=pending | PATCH /admin/guides/:id/verification | GET /admin/bookings | GET /admin/posts | GET /admin/trips | PATCH /admin/trips/:id/status | DELETE /admin/posts/:id | GET /admin/overview

Role-permission matrix

Action	Tourist	Guide	Admin
Browse trips/places	Yes	Yes	Yes
Book trip	Yes	No	No
Create/edit trip	No	Yes (own)	Yes (manage)
Verify guide	No	No (submits docs)	Yes
Manage users/bookings/posts	No	No	Yes
Approve/reject trips	No	No	Yes

10. Migration Plan (mock -> API -> Database)

- Phase 0 ☒ Stabilize Flutter (done: analyze clean; add real widget tests to replace the template counter test).
- Phase 1 ☒ Backend scaffold (NestJS, Postgres, Firebase token verification, RBAC middleware).
- Phase 2 ☒ Schema + users/auth endpoints; Flutter swaps AuthRepository internals only.
- Phase 3 ☒ Places/trips/search read endpoints; Cubits gain repositories with mock fallback.
- Phase 4 ☒ Bookings + Paymob server-side capture + My Trips sync.

- Phase 5 ☒ Guide trips/verification with server-issued Cloudinary signatures.
- Phase 6 ☒ Community (posts/stories) + notifications (FCM).
- Phase 7 ☒ Admin read screens, then admin actions; remove mock fallbacks per feature.
- Phase 8 ☒ Production hardening: secret rotation, rate limits, pagination, offline cache.

Mock-to-API mapping (excerpt)

Mock source	Used by	Future API	Future DB
mock_places	map/search/place/home/booking	GET /places	places
mockTripsJson	trips/popular/my_trips/home	GET /trips	trips
mockGuides	guides/home/booking	GET /guides	guide_profiles
MyBookingsPublic (static)	my_trips/profile	GET /bookings/my	bookings
GuideTripsStore (static)	dashboard/my_trips	GET /guide/trips	trips
mockCommunity*	community	GET /feed, POST /posts	posts/stories
mockNotificationsRaw	notifications	GET /notifications	notifications
d_mock_* (admin)	admin pages	GET /admin/*	respective tables

11. Critical Issues & Risks

- 1. Three runtime silos: tourist, guide and admin share no live data (approval buttons mutate local copies; created trips never reach tourists). Backend reconciliation of three divergent Guide/Trip models is the #1 risk.
- 2. Secrets in source: Paymob sandbox key (paymob_service.dart), Cloudinary cloud+preset (cloudinary_service.dart), Firebase/Google-Maps keys in repo ☒ rotate before release; move Paymob capture server-side.
- 3. Duplicate class AppRoutes in d_app_routes.dart (unused; importing it breaks compilation) + orphan mocks + unused firebase_storage/google_maps_flutter deps.
- 4. Template counter widget test is effectively dead; no real tests exist.
- 5. Verification can never complete (no approver); earnings/travelers static; session stores cleared on restart; admin triplykl gate is demo-only.
- 6. Unsigned Cloudinary preset + client-side folder paths let any client write anywhere under triply/ ☒ move to server-issued signatures.

12. Documentation Files

Per your instruction this deliverable is the single PDF. The 16 scoped docs/*.md files (PROJECT_OVERVIEW, ARCHITECTURE, TOURIST_FLOW, TOUR_GUIDE_FLOW, ADMIN_FLOW, CROSS_ROLE_FLOW, DATA_FLOW, BUSINESS_LOGIC, TECH_STACK, BACKEND_PLAN, DATABASE_PLAN, API_PLAN, AUTHORIZATION_PLAN, MOCK_TO_API_MIGRATION, IMPLEMENTATION_ROADMAP, KNOWN_ISSUES) were not written; say the word in build mode and I will generate them from these same chapters.

13. Recommended Next Step (one step)

ONE STEP: in build mode, say "write the docs" 📄 I will generate the 16 docs/*.md files from these chapters (analysis only, zero source changes). After that, the correct first implementation step is Phase 1: scaffold the NestJS + PostgreSQL backend with Firebase-token auth + RBAC middleware and the POST /auth/register + POST /auth/login endpoints, then swap only AuthRepository internals behind the existing AuthProvider API.